

container-integration

Container Integration

The pkg/userflow package includes TestEnvironment, a container orchestration layer that manages test infrastructure. It composes subsystems from the digital.vasic.containers module to handle container lifecycle, health checking, service registration, and event broadcasting.

TestEnvironment

Defined in container_infra.go:

```
type TestEnvironment struct {
    runtime      containers_runtime.ContainerRuntime
    compose      containers_compose.ComposeOrchestrator
    health       containers_health.HealthChecker
    registry     *containers_serviceregistry.ServiceRegistry
    eventBus     containers_event.EventBus
    logger       containers_logging.Logger
    groups       []PlatformGroup
    projectName  string
    composeFile  string
}
```

Constructor

```
env, err := userflow.NewTestEnvironment(
    userflow.WithComposeFile("docker-compose.test.yml"),
    userflow.WithProjectName("my-test"),
    userflow.WithPlatformGroups(groups),
    userflow.WithLogger(myLogger),
)
```

The constructor performs these initialization steps:

1. **Auto-detect container runtime** (Podman-first) via `containers_runtime.AutoDetect`.
2. Apply functional options.

3. **Create compose orchestrator** via `containers_compose.NewDefaultOrchestrator`.
4. **Create health checker** with built-in TCP, HTTP, and gRPC probes via `containers_health.NewDefaultChecker`.
5. **Create service registry** via `containers_serviceregistry.New`.
6. **Create event bus** with buffered delivery (capacity 64) via `containers_event.NewEventBus`.

Defaults

Setting	Default
<code>projectName</code>	<code>"userflow-test"</code>
<code>composeFile</code>	<code>"docker-compose.test.yml"</code>
<code>logger</code>	<code>containers_logging.NoopLogger{}</code>

PlatformGroup

A `PlatformGroup` defines a set of containers that run together within a resource budget:

```
type PlatformGroup struct {  
    Name          string           `json:"name"`  
    Services      []string         `json:"services"`  
    CPULimit      float64          `json:"cpu_limit"`  
    MemoryMB      int              `json:"memory_mb"`  
    HealthTargets []containers_health.HealthTarget `json:"- "`  
    ComposeFile   string           `json:"compose_file,omitempty"`  
}
```

- `Name` identifies the group (e.g., "backend", "frontend", "mobile-emulator").
- `Services` lists the compose service names in the group.
- `CPULimit` and `MemoryMB` define the resource budget for the group.
- `HealthTargets` specifies health check probes (TCP, HTTP, gRPC) for the group's services.
- `ComposeFile` optionally overrides the environment's default compose file for this group.

Lifecycle

Setup

```
err := env.Setup(ctx, group)
```

1. Resolves the compose file (group-specific or environment default).
2. Creates a ComposeProject with the project name, compose file, and service list.
3. Calls `compose.Up(ctx, project)` to start the containers.
4. Runs health checks on all `HealthTargets` for the group.
5. Returns an error if any health check fails.

Teardown

```
err := env.Teardown(ctx, group)
```

1. Resolves the compose file.
2. Calls `compose.Down(ctx, project)` to stop the containers.
3. Unregisters all group services from the service registry.

SetupAll / TeardownAll

```
err := env.SetupAll(ctx)    // starts all groups sequentially  
err := env.TeardownAll(ctx) // stops all groups in reverse order
```

`SetupAll` iterates over groups in order. `TeardownAll` iterates in reverse order to respect dependencies (services started last are stopped first). If a teardown error occurs, it is logged and the first error is returned, but all groups are attempted.

Accessor Methods

Method	Returns
<code>env.Runtime()</code>	<code>containers_runtime.ContainerRuntime</code>
<code>env.Registry()</code>	<code>*containers_serviceregistry.ServiceRegistry</code>
<code>env.EventBus()</code>	<code>containers_event.EventBus</code>

These allow challenge code to interact with the container infrastructure directly when needed.

Resource Budgets

Platform groups define CPU and memory limits. While the TestEnvironment itself does not enforce these limits at the container level (that is the responsibility of the compose file or runtime configuration), the PlatformGroup struct carries this information for:

- Documentation of expected resource usage.
- Validation by external tooling.
- Reporting in challenge outputs.

Example: Full Test Environment

```
groups := []userflow.PlatformGroup{
    {
        Name:      "backend",
        Services: []string{"api", "postgres", "redis"},
        CPULimit:   2.0,
        MemoryMB:  4096,
        HealthTargets: []containers_health.HealthTarget{
            {Name: "api", Type: "http", Address: "http://localhost:8080/health"},
            {Name: "postgres", Type: "tcp", Address: "localhost:5432"},
        },
    },
    {
        Name:      "frontend",
        Services: []string{"web"},
        CPULimit:   1.0,
        MemoryMB:  2048,
        HealthTargets: []containers_health.HealthTarget{
            {Name: "web", Type: "http", Address: "http://localhost:3000"},
        },
    },
}

env, err := userflow.NewTestEnvironment(
    userflow.WithComposeFile("docker-compose.test.yml"),
    userflow.WithProjectName("integration-test"),
    userflow.WithPlatformGroups(groups),
)
if err != nil {
    return err
}

// Start all services
err = env.SetupAll(ctx)
```

```

if err != nil {
    return err
}
defer env.TeardownAll(ctx)

// Run challenges against the started services...

```

Integration with Environment Challenges

The TestEnvironment is typically used inside EnvironmentSetupChallenge and EnvironmentTeardownChallenge:

```

env, err := userflow.NewTestEnvironment(
    userflow.WithPlatformGroups(groups),
)
if err != nil {
    return err
}

setup := userflow.NewEnvironmentSetupChallenge(
    "CH-ENV-001",
    func(ctx context.Context) error {
        return env.SetupAll(ctx)
    },
    120*time.Second,
)

teardown := userflow.NewEnvironmentTeardownChallenge(
    "CH-ENV-TEARDOWN",
    func(ctx context.Context) error {
        return env.TeardownAll(ctx)
    },
)

```

Containers Module Dependencies

The TestEnvironment uses these subsystems from digital.vasic.containers:

Subsystem	Import Path	Purpose
Runtime	containers/pkg/runtime	Auto-detect Podman or Docker
Compose	containers/pkg/compose	Orchestrate multi-container environments
Health	containers/pkg/health	TCP/HTTP/gRPC health probes

Subsystem	Import Path	Purpose
Service Registry	containers/pkg/serviceregistry	Track running services
Event Bus	containers/pkg/event	Broadcast container lifecycle events
Logging	containers/pkg/logging	Structured logging